

JavaScript Function

HCL (Higher Coding Language) JavaScript

Arrow Function

Arrow function expressions is a new syntax to writing ordinary function as expressions.

first.html

```
<html>
<head>
  <title>Arrow Function</title>
  <script>
    // wirte ordanry function
    function add(no1,no2)
    {
      var sum=no1+no2;
      return sum;
    }
    document.write("Sum="+add(10,20)+"<br>")
    // wirte arrow function
    var data=(no1,no2) => {
      var sum=no1+no2;
      return sum;
    }
    document.write("Sum="+data(10,20)+"<br>")
    // wirte arrow function with one line
    var data=(no1,no2)=>no1+no2;
    document.write("Sum="+data(10,20))
  </script>
</head>
<body>
</body>
</html>
```

Output

HCL (Higher Coding Language) JavaScript

← → ↻ ⓘ 127.0.0.1:5500/abc.html

Sum=30

Sum=30

Sum=30

For each loop

For each loop is the advanced version of for loop.

first.html

```
<html>
  <head>
    <title>For Each loop</title>
    <script>
      var arr=[10,20,30,40,50]
      arr.forEach((item)=>
      {
        document.write(item+"<br>");
      })
    </script>
  </head>
  <body>
  </body>
</html>
```

Here for each loop pass an arrow function or callback function which take a parameter item, which assign one by one value from array.

first.html

```
<html>
  <head>
    <title>For Each loop</title>
    <script>
```

HCL (Higher Coding Language) JavaScript

```
var arr=[10,20,30,40,50]
var data= arr.forEach((item)=>
{
    return item;

})
document.write(data);
</script>
</head>
<body>
</body>
</html>
```

Output



Callback function

If we pass a function into another function as parameter is called callback function and function which we pass function is called high order function.

first.html

```
<html>
<head>
    <title>Callback Function</title>
    <script>
        function hello()
        {
```

HCL (Higher Coding Language) JavaScript

```
        document.write("Hello")
    }
    function add(no1,no2,callback)
    {
        document.write(no1+no2)
        callback()
    }
    add(10,20,hello)
</script>
</head>
<body>
</body>
</html>
```

Output

30Hello

```
<!DOCTYPE html>
<html lang="en">
<head>
    <title>Document</title>
</head>
<body>
    <script>
        var add=function(num1,num2)
        {
            return num1+num2
        }

        var sumofthreenum=function(num1,num2,num3,sumoftwonum)
        {
            var sum1=sumoftwonum(num1,num2)
            return sumoftwonum(sum1,num3)
        }
    </script>

```

HCL (Higher Coding Language) JavaScript

```
    }  
    document.write("Addition="+sumofthreenum(10,20,30,add))  
  </script>  
</body>  
</html>
```

Output

Addition=60

Map

map generally use to read JSON Objects.

first.html

```
<html>  
  <head>  
    <title>Map</title>  
    <script>  
      var data=[{rollno:101,name:"Ram",marks:45},  
                 {rollno:102,name:"Syam",marks:55},  
                 {rollno:103,name:"Mohan",marks:65}]  
      data.map((item)=>  
      {  
        document.write(item.rollno+" "+item.name+"  
"+item.marks+"<br>")  
      })  
    </script>  
  </head>  
  <body>  
  </body>  
</html>
```

Output

HCL (Higher Coding Language) JavaScript

← → ↻ ⓘ 127.0.0.1:5500/first.html

101 Ram 45
102 Syam 55
103 Mohan 65

The difference between for each and map is that in for each loop we cannot assign value in variable as return but in map we can do.

first.html

```
<html>
  <head>
    <title>For Each loop</title>
    <script>
      var arr=[10,20,30,40,50]
      var data= arr.forEach((item)=>
      {
        return item;
      })
      document.write(data);
    </script>
  </head>
  <body>
  </body>
</html>
```

Output

undefined

first.html

```
<html>
```

HCL (Higher Coding Language) JavaScript

```
<head>
  <title>Map</title>
  <script>
    var arr=[10,20,30,40,50]
    var data=arr.map((item)=>
    {
      return item;
    })
    document.write(data);
  </script>
</head>
<body>
</body>
</html>
```

Output

10,20,30,40,50

Nested objects

first.html

```
<html>
  <head>
    <title>Nested Objects</title>
    <script>
      var data={
        rollno:103,
        name:"Mohan",
        marks:65,
        add:{city:"Indore",pin:452020}
      }
      document.write(data.name+" "+data.add.city)
    </script>
  </head>
  <body>
```


HCL (Higher Coding Language) JavaScript

```
</body>  
</html>
```

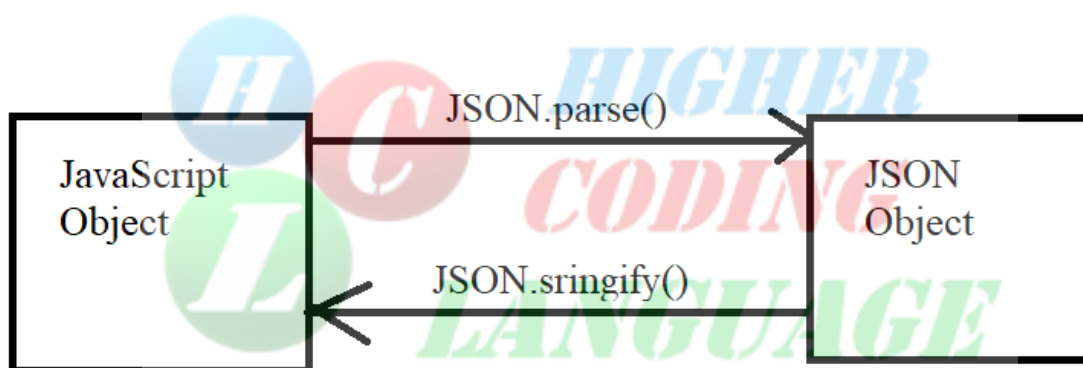
Output

Mohan Indore

Javascript Object and JSON Object

JavaScript object: `var data={rollno:101,name:"Ram"}`

JSON object: `var data={"rollno":101,"name":"Ram"}`



Autoboxing

Automatic conversion of primitive data types to objects.

first.html

```
<html>  
  <head>  
    <title>Autoboxing</title>  
    <script>  
  
      var name="Arvind Choudhary"  
      document.write(name.length) //converting str to str object
```

HCL (Higher Coding Language) JavaScript

```
document.write("<br>")
document.write(name.toUpperCase())//converting str to str
object
</script>
</head>
<body>
</body>
</html>
```

Output

16
ARVIND CHOUDHARY

Filter

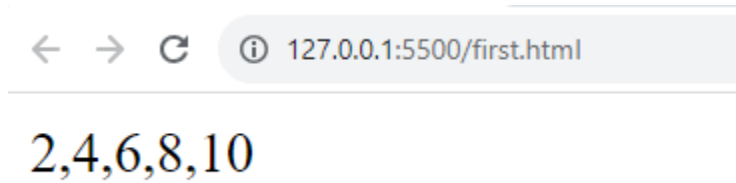
filter generally use to filter some data from group of elements.

first.html

```
<html>
<head>
  <title>Filter</title>
  <script>
    var data=[1,2,3,4,5,6,7,8,9,10]
    var i= data.filter((item)=>
    {
      return item%2==0
    })
    document.write(i)
  </script>
</head>
<body>
</body>
</html>
```

HCL (Higher Coding Language) JavaScript

Output



Reduce Method

first.html

```
<html>
  <head>
    <title>Reduce Method</title>
    <script>
      var productPrice=[100,200,300,400,500]
      var totalPrice=productPrice.reduce((accu,curelem)=>{
        return accu+curelem
      },0)
      document.write(totalPrice)
    </script>
  </head>
  <body>
  </body>
</html>
```

Output

1500

Function

HCL (Higher Coding Language) JavaScript

Function is collections of instructions which is use for reusability.

Advantages of Function

- Code reusability
- Easy maintenance
- reduce complexity
- categorization of program
- remove boilerplate code

Simple Function

Simple function use when input is same.

first.html

```
<html>
  <head>
    <title>Simple Function</title>
    <script>
      function add()
      {
        var a=10;
        var b=20;
        var addition=a+b;
        document.write("Addition="+addition);
      }
      add();
    </script>
  </head>
  <body>
  </body>
</html>
```

We can also write function by anonymous (function without name) function. It means variable addition working like as objects here.

first.html

```
<html>
```

HCL (Higher Coding Language) JavaScript

```
<head>
  <title>Document</title>
  <script>
    var add=function()
    {
      var a=10;
      var b=20;
      var addition=a+b;
      document.write("Addition="+addition);
    }
    add();
  </script>
</head>
<body>

</body>
</html>
```

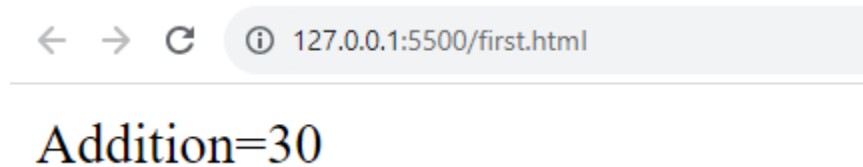
We can also write function using arrow function

first.html

```
<html>
<head>
  <title>Document</title>
  <script>
    var add=()=>
    {
      var a=10;
      var b=20;
      var addition=a+b;
      document.write("Addition="+addition);
    }
    add();
  </script>
</head>
<body>
</body>
</html>
```

HCL (Higher Coding Language) JavaScript

Output



Limitation of simple function is that we get same result on every input.

Parameterized or argument Function

Parameterized function use when we want to get different output on different input.

first.html

```
<html>
  <head>
    <title>Parameterized Function</title>
    <script>
      function add(a,b)
      {
        var addition=a+b;
        document.write("Addition="+addition+"<br>");
      }
      add(10,20)
      add(30,20)
    </script>
  <body>
  </body>
</html>
```

Output

HCL (Higher Coding Language) JavaScript

← → ↻ ⓘ 127.0.0.1:5500/first.html

Addition=30

Addition=50

Function as expression

first.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Function as expression</title>
  <script>
    var result=function add(a,b)
    {
      var addition=a+b;
      document.write("Addition="+addition+"<br>");
    }

    result(10,20)
  </script>
</head>
<body>

</body>
</html>
```

Output

← → ↻ ⓘ 127.0.0.1:5500/first.html

Addition=30

HCL (Higher Coding Language) JavaScript

Anonymous Function

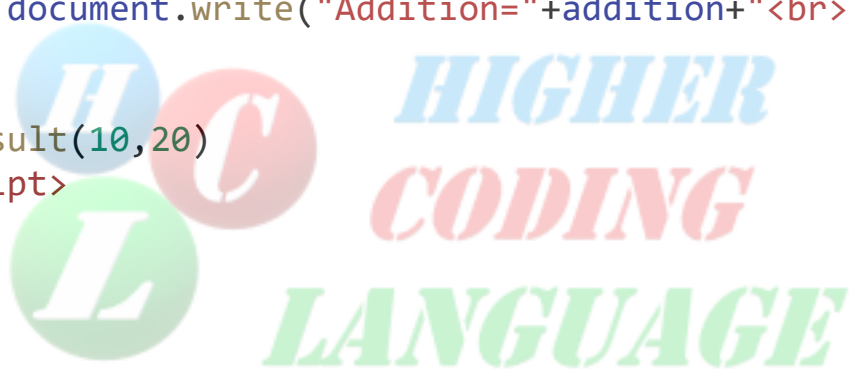
first.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Anonymous Function</title>
  <script>

    var result=function(a,b)
    {
      var addition=a+b;
      document.write("Addition="+addition+"<br>");
    }

    result(10,20)
  </script>
</head>
<body>

</body>
</html>
```

The logo for HCL Higher Coding Language is centered in the background. It features the letters 'H', 'C', and 'L' in large, stylized fonts, each inside a colored circle (blue, red, and green respectively). To the right of these letters, the words 'HIGHER CODING LANGUAGE' are written in a bold, sans-serif font, with 'HIGHER' in blue, 'CODING' in red, and 'LANGUAGE' in green.

Arrow Function

first.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Arrow Function</title>
  <script>
    var result=(a,b)=>
    {
      var addition=a+b;
      document.write("Addition="+addition+"<br>");
    }
  </script>
</head>
<body>

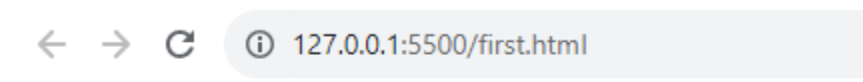
</body>
</html>
```


HCL (Higher Coding Language) JavaScript

```
        result(10,20)
    </script>
</head>
<body>

</body>
</html>
```

Output



Addition=30

Function with return type

Return type function use when we want to get inside function data at outside of the function.

first.html

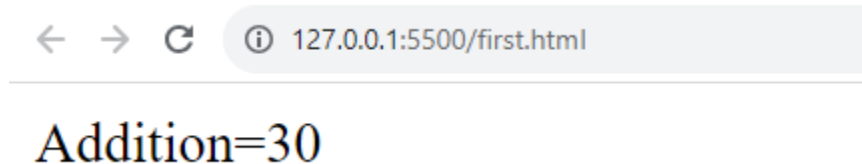
```
<html>
  <head>
    <title>Function with return type</title>
    <script>
      function add(a,b)
      {
        var addition=a+b;
        return addition;

      }
      document.write("Addition="+add(10,20));
    </script>
  <body>
</body>
```

HCL (Higher Coding Language) JavaScript

```
</html>
```

Output



But there a limitation of return, that we return only one value at a time. But if we want to get multiple data we have to assign local value globally.

first.html

```
<html>
  <head>
    <title>Function with return type</title>
    <script>
      var no1,no2;
      function add(a,b)
      {
        no1=a;
        no2=b;
      }
      add(10,20)
      var sum=no1+no2;
      document.write("Addition="+sum);
    </script>
  <body>
  </body>
</html>
```

Output

HCL (Higher Coding Language) JavaScript

← → ↻ ⓘ 127.0.0.1:5500/first.html

Addition=30

Global and Local Variable

There are two types of variable exist in JS Global and local variable. Global variable scope is within the program but local variable scope within a function.

first.html

```
<html>
<head>
  <title>Global and Local Var</title>
  <script>
    var a = 10; //global variable
    function hello(b) //local variable
    {
      var c = 20; //local variable
    }
    hello(10)
    document.write("a="+a+"<br>")
    document.write("b="+b+"<br>")
    document.write("c="+c+"<br>")
  </script>
<body>
</body>
</html>
```

Output

a=10

var VS let

HCL (Higher Coding Language) JavaScript

	Var	Let
1	The var is a keyword that is used to declare a variable	The let is also a keyword that is used to declare a variable.
2	Syntax -: var name = "Ram";	Syntax -: let name = "Ram";
3	The variables that are defined with var statement have function scope.	The variables that are defined with let statement have block scope.
4	We can declare a variable again even if it has been defined previously in the same scope.	We cannot declare a variable more than once if we defined that previously in the same scope.
5	var is an ECMAScript1 feature.	let is a feature of ES6.

Function Calling Function

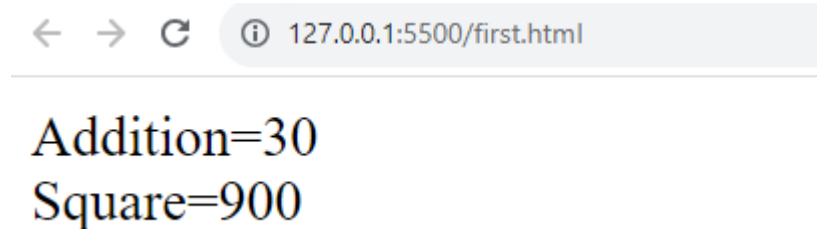
first.html

```
<html>
<head>
  <title>Function Calling Function</title>
<script>
  function add(a,b)
  {
    var addition=a+b;
    document.write("Addition="+addition+"<br>");
    sq(addition);
  }
  function sq(addition)
  {
    var square=addition*addition;
    document.write("Square="+square);
  }
  add(10,20)
```

HCL (Higher Coding Language) JavaScript

```
</script>
<body>
</body>
</html>
```

Output



Single Tasking Function

In ideal condition should be perform single task by a single function to increase the performance of the program.

first.html

```
<html>
<head>
  <title>Single Tasking Function</title>
  <script>
    function add(a,b)
    {
      var addition=a+b;
      document.write("Addition="+addition+"<br>");
    }
    add(10,20)
  </script>
<body>
</body>
</html>
```

Above the program performing 3 task at a time

1 getting value

HCL (Higher Coding Language) JavaScript

2 execute the input

3 showing result

So same program we can write by single tasking function.

first.html

```
<html>
  <head>
    <title>Single Tasking Function</title>
    <script>
      var no1,no2,sum;
      function get(a,b)
      {
        no1=a;
        no2=b;
      }
      function add()
      {
        sum=no1+no2;
      }
      function show()
      {
        document.write("addition="+sum);
      }
      get(10,20);
      add();
      show();
    </script>
  <body>
  </body>
</html>
```

Output

HCL (Higher Coding Language) JavaScript

← → ↻ ⓘ 127.0.0.1:5500/first.html

addition=30

Recursion Function

If a function called itself is called recursion function.

first.html

```
<html>
  <head>
    <title>Recursion Function</title>
    <script>
      var count=0;
      function hello()
      {
        count++;
        document.write("Hello="+count+"<br>");
        hello()
      }
      hello();
    </script>
  <body>
  </body>
</html>
```

Output

HCL (Higher Coding Language) JavaScript

127.0.0.1:5500/first.html

Hello=1
Hello=2
Hello=3
Hello=4
Hello=5
Hello=6
Hello=7
Hello=8
Hello=9
Hello=10
Hello=11
Hello=12
Hello=13
Hello=14

Factorial
first.html

```
<html>
  <head>
    <title>Factorial</title>
    <script>

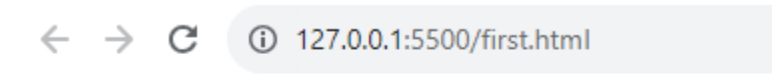
      function fact(no)
      {
        if(no==1)
        {
          return 1;
        }
        else{
          return no*fact(no-1);
        }
      }

      document.write("Factorial="+fact(5));
    </script>
  <body>
  </body>
```


HCL (Higher Coding Language) JavaScript

```
</html>
```

Output



Factorial=120

Immediately invoked function expression (IIFE)

first.html

A function call automatically is called immediate invoked function expression.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>IIFE</title>
  <script>
    (function add(a,b)
    {
      var addition=a+b;
      document.write("Addition="+addition+"<br>");
    })(10,20)
  </script>
</head>
<body>

</body>
</html>
```

Output

HCL (Higher Coding Language) JavaScript

← → ↻ ⓘ 127.0.0.1:5500/first.html

addition=30



Higher Coding Language

Live Project Training With 100% Placement Assistance

S no	Internship	Content	Duration
1	C,C++ with web development	C,C++,html, CSS ,JS & Project	2 Months
2	Web Development	html, CSS ,JS,Jquery,Bootstrap & Project	2 Months
3	C with DSA	C and DSA	2 Months
4	C++ with DSA	C++ and DSA	2 Months
5	Java with DSA	Java and DSA	2 Months
6	Java With Mysql	Core java and Mysql	2 Months
7	Front end	html, CSS ,JS, React JS & Project	2 Months
8	Back end	Node JS & Project	2 Months
9	Core java with Web Development	html, CSS ,JS,Jquery,Bootstrap,Core Java & Project	3 Months
10	Python with Web Development	html, CSS ,JS,Jquery,Bootstrap,Python & Project	3 Months
11	MERN Full Stack	html, CSS ,JS,Jquery,Bootstrap,Node JS & Live Project	6 Months
12	Java Full Stack	html, CSS ,JS,Jquery,Bootstrap,Core Java,JDBC,JSP,Servlet & Live Project	6 Months
13	Python Full Stack	html, CSS ,JS,Jquery,Bootstrap,Core Python,Advanced Python,Django & Live Project	6 Months
14	Android	Core Java and Android	4 Months
15	Fluter	Dart and Flutter	4 Months

HCL (Higher Coding Language) JavaScript

16	Collage Minor Project	In any Technology	NA
17	Collage Major Project	In any Technology	NA
18	Python With ML	Core Python, Advanced Python,Numpy,Pandas, Mitlab, Seaborn & ML Algorithms	6 months
19	Python With AI	Core Python, Advanced Python,Numpy,Pandas, Mitlab, Seaborn & AI Algorithms	6 months
20	Python Data science	Core Python, Advanced Python,Numpy,Pandas, Mitlab, Seaborn & Data Science Algorithms	6 months
20	Python Data Analytics	Core Python, Advanced Python,Numpy,Pandas, Mitlab, Seaborn ,mysql and power BI	6 months

Our Recent Placements

Name	Photo	Company	PKG
Khusbu Dubey		TCS	3.59 LPA
Prashant Shukla		Infosys	3.50 LPA
Tanuja Patidar		BestPeers	2.0 LPA
Sneha Gupta		Cyber Intant	NA

HCL (Higher Coding Language) JavaScript

Rohan Sisodiya		GeeCom India	1.2 LPA
Kanchan pandey		SheThink	1.8 LPA
Sachin Choudhary		Nokia	4.0 LPA
Shaikhar parmar		Capgemini	3.80 LPA
Lakhan Patel		TCS	5.00 LPA
Kundan Mandloi		Maveric Systems	2.88 LPA
Yoegsh		Deloitte	3.20 LPA

HCL (Higher Coding Language) JavaScript

Dheeraj Patel		Cognizant	4.50
Rahul		Digiprima	1.9 LPA
Jayendra		Assistant	2.4 LPA
Sachin		Codernaline	1.60 LPA
Sanchit		Avery Bit	1.80 LPA
Sumit Chandravanshi		Geecom India	1.20 LPA

HCL (Higher Coding Language) JavaScript

Ajay Sailani				Avery Bit	1.20 LPA
--------------	--	---	--	-----------	----------

Other Facility

- 100% Placement Assistance
- Live Coding
- Highly Experience Industrial Trainer
- Work on Live Project
- Free Printed Notes
- 2 Days Free Demo
- Mock Interview
- Certificate of Internship
- 1000 Rs Referral Amount
- Corporate Environment
- Lab Facility Available
- May Be Contact for Placement
- Offline Training also available

Contact No – 82368 09542, 75662 99542

Add - 109,208 Prem Plaza, Ashok Nagar,Bhwarkua, Indore – 452001 (M.P.)